A Practical Guide to Spatial Data

UNIVERSITÀ
DI TORINO

Outline

1. **Coordinates, CRS & Reprojection**: How spatial data gets a position on Earth.
2. **Types of Spatial Data**: Vector, raster, and network representations.
3. **Vector Data**: Geometries, attributes, CRS, and GeoPandas examples.
4. **Raster Data**: Grids, pixels, and surfaces.
5. **Network Data**: Nodes, edges, and connectivity.
6. **Working Environment & Wrap-up**: The reproducible Python stack for the notebooks.

1. Coordinates, CRS & Reprojection

What a spatial dataset contains

A spatial dataset is a table plus the information needed to locate, interpret, and measure its observations.

Data components

- **Attributes**: variables measured for each observation
- **Geometry or grid**: point, line, polygon, cell, or network edge
- **CRS**: how coordinates are tied to Earth
- **Spatial support**: the unit represented by each observation

Analysis components

- **Scale and resolution**: how much space each observation represents
- **Extent**: the spatial domain covered by the dataset
- **Time stamp or period**: when observations apply
- **Metadata**: source, processing, uncertainty, and validity checks

Coordinates are not enough

A coordinate pair such as **(11.34, 44.49)** is only meaningful when we know the coordinate reference system (CRS). Without a CRS, the numbers do not define a position, a unit of measurement, or a valid distance.

Coordinates

- Numeric position in a coordinate system
- Often written as **(x, y)** or **(lon, lat)**
- Units may be degrees, metres, feet, or grid cells

CRS

- Defines the datum, axes, units, and projection
- Connects numbers to Earth
- Determines which measurements are valid

Professional habit: every spatial dataset should carry a CRS, and every measurement should state the CRS in which it was made.

What is a Coordinate Reference System (CRS)?

A CRS defines how coordinates map to locations on Earth. It is not metadata in the decorative sense; it controls geometry, distance, area, overlays, and joins.

Geographic CRS

- Uses angular coordinates on an ellipsoid or datum.
- Units are in degrees.
- Appropriate for global location; unsuitable for planar distance or area.
- **Example**: `EPSG:4326` (WGS 84)

Projected CRS

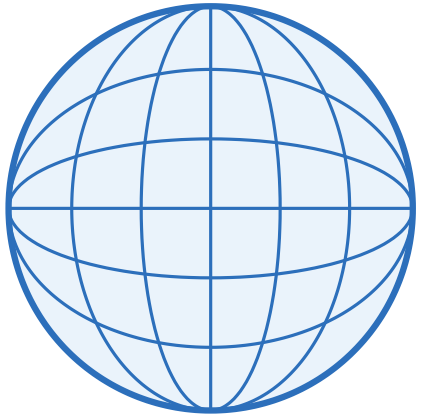
- Transforms Earth to a flat coordinate plane.
- Units are in meters or feet.
- **For measurement**: use UTM zones or an equal-area CRS suited to the study area.
- **For display only**: `EPSG:3857` (Web Mercator) is for web tiles — conformal, not equal-area. *Do not measure on it*.

Geographic to projected CRS

Reprojection moves coordinates from an angular Earth-referenced system to planar coordinates suitable for measurement.

Geographic CRS

degrees · EPSG:4326



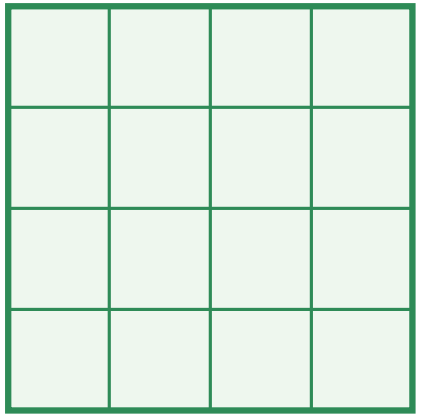
lat / lon on a sphere

to_crs(...)

flatten the Earth

Projected CRS

meters · EPSG:3857 / UTM



x / y on a plane

Store/exchange: lon/lat is common. **Measurement/modeling:** use a suitable projected CRS.

What a CRS contains

A CRS bundles several decisions that must agree across datasets before analysis.

Reference system

- **Datum / reference frame:** how the Earth is modeled and anchored.
- **Ellipsoid:** mathematical shape approximating the Earth.
- **Epoch:** relevant for high-precision data on a moving Earth.

Coordinate system

- **Axes:** order and direction, e.g. longitude/latitude or easting/northing.
- **Units:** degrees, metres, feet.
- **Projection:** required for planar coordinates.

EPSG codes, such as **EPSG:4326** or **EPSG:32633**, are compact identifiers for standard CRS definitions. They are convenient, but the analyst still has to know what the code means for measurement.

Geographic vs. projected CRS

Geographic and projected CRS are both valid, but they support different tasks.

Question	Appropriate CRS	Reason
Store global locations	Geographic (`EPSG:4326`)	Portable lon/lat coordinates
Web basemap display	Web Mercator (`EPSG:3857`)	Tile ecosystem standard
Distance or buffers	Local projected CRS	Linear units and controlled distortion
Area summaries	Equal-area projection	Preserves area relationships
Local fieldwork/GPS	Dataset CRS + metadata	Avoid silent shifts between frames

Rule: longitude/latitude in degrees is usually fine for storage and display,
but not for planar measurement.

Distortion is a design choice

Every projection distorts something because a curved Earth is being represented on a plane. The question is which distortion your analysis can tolerate.

- **Conformal projections** preserve local angles and shape; Web Mercator supports web display but distorts area.
- **Equal-area projections** preserve area; use them for density, exposure, and areal summaries.
- **Equidistant projections** preserve selected distances for analyses tied to specific origins or lines.
- **Local projected CRS** such as UTM often provide good distance and area behavior over a limited region.

Projection choice is part of the research method, not a cosmetic setting.

Axis order and units

Many spatial bugs are not mathematical; they are bookkeeping errors.

Axis order

- GIS practice often writes **(lon, lat)**.
- Some CRS definitions formally list latitude before longitude.
- Python tools often expose safeguards such as **always_xy=True**.

Units

- **EPSG:4326**: degrees.
- Projected CRS: usually metres or feet.
- Buffering by **500** means different things depending on CRS units.

Always inspect the CRS and confirm units before measuring distance, area, or neighbourhoods.

CRS workflow in GeoPandas

Three operations should be distinct in your mind and in your code.

- `gdf.crs` : inspect the CRS currently attached to the data.
- `gdf.set_crs(...)` : assign a missing CRS **without moving coordinates**.
- `gdf.to_crs(...)` : transform coordinates into another CRS.

Use `set_crs` only when reliable metadata identifies the current coordinate system. Use `to_crs` when coordinates must be expressed in another CRS for measurement, overlay, or storage.

CRS failure modes to recognize

- Missing CRS: **`.crs is None`**; analysis should stop until metadata is recovered.
- Wrongly assigned CRS: coordinates appear in the wrong country or ocean.
- Degree-based measurement: distances, buffers, or areas computed in lon/lat.
- Mixed CRS overlay: layers do not line up or joins silently fail.
- Axis swap: latitude and longitude reversed.
- Web Mercator misuse: display CRS treated as an analysis CRS.

CRS errors are serious because they can produce plausible-looking maps with wrong measurements.

Choosing a CRS for analysis

Choose the CRS from the analytical task and spatial extent.

1. **Identify the task:** storage, display, distance, area, overlay, routing.
2. **Identify the extent:** city, country, continent, global.
3. **Choose a CRS with appropriate units and distortion properties.**
4. **Reproject all layers used in the same operation.**
5. **Document the CRS used for reported measurements.**

For a small study area, a suitable UTM zone is often adequate. For regional or global area comparisons, use an equal-area CRS selected for the region.

Three CRS rules for practice

- `gdf.set_crs("EPSG:4326")` : **Assigns** a CRS to data that has none. It **does not change the data**. Only use it if you are 100% certain what the CRS should be.
- `gdf.to_crs("EPSG:3857")` : **Reprojects** the data, creating new coordinate values. Use this to transform data from one CRS to another.
- Never measure distance, area, or buffers in a geographic CRS unless the method explicitly handles geodesic measurement.

Library Guide: `pyproj`

`pyproj` wraps the **PROJ** library used by GeoPandas and Rasterio for CRS transformations. Use it directly for raw coordinate arrays, CRS inspection, or cases where building a full GeoDataFrame is unnecessary.
Create a `Transformer.from_crs(..., always_xy=True)` once and reuse it. The `always_xy=True` argument fixes `(lon, lat)` input order and avoids axis-order ambiguity.

Low-Level Reprojection with `pyproj` (setup)

Use `pyproj` directly when the input is a coordinate array rather than a spatial table. GeoPandas uses the same CRS machinery through `to_crs`.

```
from pyproj import CRS, Transformer

# 1) Define the source and destination CRS
# From WGS 84 (lon/lat) to Web Mercator (meters)
source_crs = CRS("EPSG:4326")
dest_crs = CRS("EPSG:3857")

# 2) Create a transformer
# always_xy=True ensures (lon, lat) -> (x, y) order
transformer = Transformer.from_crs(source_crs, dest_crs, always_xy=True)
```

Low-Level Reprojection with `pyproj` (transform)

```
# 3) Define a lon/lat point (San Diego)
lon, lat = -117.16, 32.71

# 4) Transform the point
x, y = transformer.transform(lon, lat)

print(f"Original (lon, lat): ({lon}, {lat})")
print(f"Projected (x, y): ({x:.2f}, {y:.2f})")
```

> **Lesson —** for a raw coordinate array, reach for **`pyproj.Transformer`** directly — build it once with **`always_xy=True`** and reuse it, no GeoDataFrame required.

CRS & Reprojection: Further Learning

- **Pyproj Documentation**: The library that powers CRS transformations in Python.
  - https://pyproj4.github.io/pyproj/stable/
- **GeoPandas User Guide on CRS**: Practical examples and explanations.
  - https://geopandas.org/en/stable/docs/user_guide/projections.html
- **Pyproj Docs on CRS**: Understanding CRS objects.
  - https://pyproj4.github.io/pyproj/stable/api/crs/crs.html

2. Types of Spatial Data

Representation is a modeling choice

The same phenomenon can be represented in different ways. That choice determines what can be measured, joined, aggregated, modeled, and validated.

Phenomenon	Possible representation	Consequence
Air pollution	Raster surface, sensor points, zones	Changes interpolation, exposure, and uncertainty
Mobility	Points, trajectories, street network	Changes distance, accessibility, and routing
Disease risk	Individuals, neighborhoods, grid cells	Changes privacy, aggregation bias, and inference

Choose the representation from the research question before choosing a file format, library, or database.

A Quick Tour of Spatial Data Models

The first question on any project is which data model fits: **vector** for discrete objects with crisp edges, **raster** for a continuous field sampled on a grid.

Vector

Represents discrete objects with exact boundaries.

- **Points**: Locations (cities, sensors)
- **Lines**: Paths (rivers, roads)
- **Polygons**: Areas (countries, parks)

Use for: Administrative boundaries, infrastructure, points of interest.

Raster

Represents continuous phenomena on a grid of cells (pixels).

- Each pixel has a value.
- Examples: Elevation, temperature, satellite imagery.

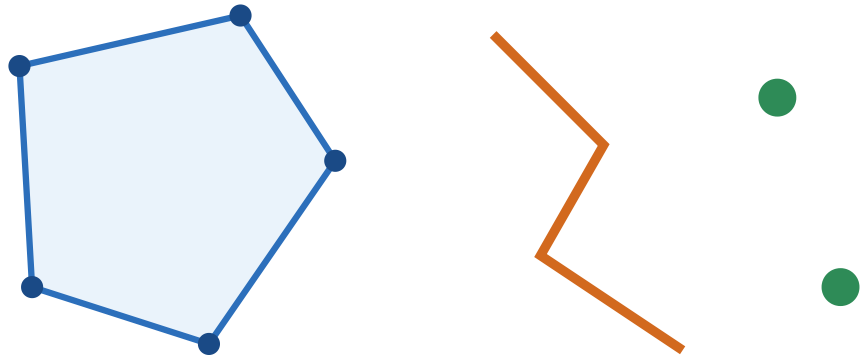
Use for: Environmental data, imagery, surface analysis.

Networks (nodes + edges) are covered later for routing. *3D **point clouds** (LiDAR) are a fourth model, but out of scope in this course.*

Vector and raster: visual comparison

Vector

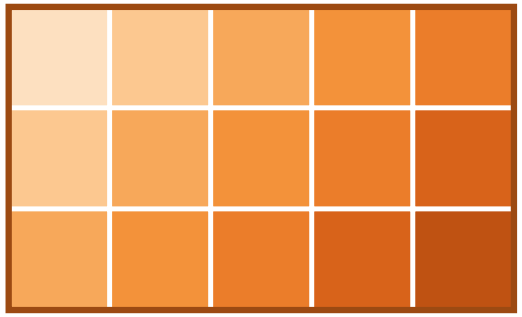
discrete objects · exact boundaries



points · lines · polygons

Raster

continuous surface · grid of cells



each cell holds one value

Vector data store discrete geometries; raster data store values in cells. This choice determines the file formats, indexes, transformations, and algorithms available for analysis.

3. Vector Data

Working with Points, Lines, and Polygons

Vector data is ideal for representing features with clear, defined shapes.

- **Points** represent events, samples, sensors, addresses, and facilities.
- **Lines** represent paths, rivers, streets, pipelines, and boundaries.
- **Polygons** represent areas such as parcels, census units, land cover, and administrative regions.

Each feature has geometry plus attributes. The geometry enables spatial operations; the attributes carry the variables used for description, joining, modeling, and interpretation.

GeoPandas as the vector-data interface

GeoPandas is not a spatial data model. It is the Python interface used in this course to work with vector data after the conceptual choices have been made.

It represents

- A table of attributes
- One active geometry column
- CRS metadata
- Spatial methods attached to the table

It supports

- Reading and writing vector files
- CRS inspection and reprojection
- Spatial joins, overlays, dissolve
- Geometry cleaning and validation
- Feature engineering before modeling

Library Guide: `geopandas`

`geopandas` is the main vector-data interface for Python analysis. A `GeoDataFrame` is a `pandas.DataFrame` with a geometry column, so filtering, grouping, joins, and plotting work alongside spatial methods such as `to_crs`, `sjoin`, overlay, dissolve, and `.plot()`.
For correctness, verify `gdf.crs` on load and reproject before measuring area or distance. For performance, use `.sindex`, clipping, or bounding-box filters before expensive spatial joins.
GeoPandas connects to **Shapely / GEOS** for geometry operations, **Pyogrio / GDAL** for vector I/O, **Pyproj / PROJ** for CRS transformations, and **PostGIS / GeoParquet / DuckDB** when data need stronger storage or query systems.

GeoPandas example: CRS and area

This example uses a vector dataset, checks its CRS, reprojects it to a metric CRS, and calculates area. The CRS choice is part of the measurement method.

```
import geopandas as gpd

gdf = gpd.read_file("data/berlin-neighbourhoods/berlin-neighbourhoods.geojson")
print(f"Original CRS: {gdf.crs}")

gdf_utm = gdf.to_crs("EPSG:32633")
print(f"New CRS: {gdf_utm.crs}")

gdf_utm["area_sqkm"] = gdf_utm.geometry.area / 1_000_000
print(gdf_utm[["neighbourh", "area_sqkm"]].head())
```

> **Lesson —** reproject to a *metric* CRS before measuring area or distance — computing `.area` on degrees returns meaningless numbers.

> **Notebook:** notebooks/00_vector_data.ipynb

Library Guide: `shapely`

`shapely` is the geometry engine underneath GeoPandas. It provides geometry objects and operations such as `buffer`, `union`, `intersection`, `contains`, and `intersects`. Use it directly for per-geometry logic or geometry repair with `make_valid()`.
Metric operations require a projected CRS. Repeated predicate tests against one geometry can use a prepared geometry for better performance.

Library Guide: `fiona`

fiona is a GDAL/OGR wrapper for vector file I/O. It opens layers, exposes schemas and CRS metadata, and can stream features without loading a full dataset into memory. Use it for lower-level file control, legacy encodings, or custom driver options.
Since GeoPandas 1.0, **pyogrio** is the default vectorized I/O engine behind **read_file** / **to_file**. Fiona remains relevant for feature-by-feature streaming and legacy files.

Vector Formats & Trade-offs

File format controls read performance, attribute preservation, interoperability,
and storage limits. The three formats below appear often in applied spatial work.

- **Common Formats & Trade-offs**:
  - **GeoPackage (`.gpkg`)**: **Best choice**. An open standard, single-file database. Fast, flexible, and robust.
  - **Shapefile (`.shp`)**: Legacy format. Avoid if possible. Multi-file, column name limits, and other issues.
  - **GeoJSON (`.geojson`)**: Text-based web exchange format. Not efficient for large datasets.

GeoPackage (`.gpkg`)

A GeoPackage is a single SQLite file that can hold many layers plus their spatial indexes, storing CRS and attributes without loss. Treat it as your default working format: write once, read any layer back by name.

```python
import os, geopandas as gpd

os.makedirs("outputs", exist_ok=True)
# Write to a new GeoPackage layer
gdf = gpd.read_file("data/texas/texas.geojson")
gdf.to_file("outputs/data.gpkg", layer="texas", driver="GPKG")

# Read a specific layer back
gpkg = gpd.read_file("outputs/data.gpkg", layer="texas")
print(len(gpkg), gpkg.crs)
# → 254 EPSG:4326   (round-trip preserves all 254 counties and the CRS)
```

> **Lesson —** make GeoPackage your system of record — one file, many layers, lossless CRS and attributes, spatial index included.

Shapefile (`.shp`)

The Shapefile is a 1990s format that travels as a bundle of sidecar files (**`.shp/.shx/.dbf/.prj`**) and silently truncates field names to 10 characters. Treat it as an interchange format you read from, not the system of record you write to.

```
import os, geopandas as gpd

os.makedirs("outputs", exist_ok=True)
gdf = gpd.read_file("data/texas/texas.geojson")
gdf[["NAME", "STATE_NAME", "HR90", "geometry"]].to_file("outputs/texas.shp")

shp = gpd.read_file("outputs/texas.shp")  # writes .shp/.shx/.dbf/.prj sidecars
print(list(shp.columns))
# → ['NAME', 'STATE_NAME', 'HR90', 'geometry']
# (longer attribute names are cut to 10 chars, so "population_density" becomes
#  "population"; name collisions then get a numeric suffix, e.g. "neighbourh"
#  and "neighbou_1" in the bundled Berlin data)
```

> **Lesson —** treat Shapefiles as read-only interchange; the 10-character field-name limit silently truncates attributes.

Validate and fix geometries

Self-intersections and slivers make predicates and overlays misbehave, so screen for them on load. Prefer **make_valid()** — it repairs geometry topologically — over the old **buffer(0)** trick, which can quietly drop or distort parts.

```
import geopandas as gpd

gdf = gpd.read_file("data/berlin-neighbourhoods/berlin-neighbourhoods.geojson")
invalid = ~gdf.is_valid
gdf.loc[invalid, "geometry"] = gdf.loc[invalid, "geometry"].make_valid()
print(f"Fixed {int(invalid.sum())} invalid geometries")
# → Fixed 0 invalid geometries   (this Berlin layer is already clean)
```

> **Lesson —** screen **is_valid** on load and repair with **make_valid()**; never assume a downloaded layer is topologically clean.

Spatial index & performance

Spatial joins are quadratic if you test every feature against every other one. A spatial index turns that into a fast bounding-box lookup, so a few habits keep large joins from grinding to a halt.

- Use **`gdf.sindex`** (Shapely 2.x STRtree, GEOS-backed) to prefilter by bounding box.
- Avoid O(N×M) spatial joins by pre-clipping candidates.
- Read only needed columns; write GeoPackage for multi-layer outputs.

Aggregate and dissolve

dissolve is **groupby** for geometry: it merges all features sharing a key into one shape while aggregating their attributes. Here we melt 254 Texas counties up into the single state outline — and only measure area *after* reprojecting to an equal-area CRS, never in degrees.

```
import geopandas as gpd

gdf = gpd.read_file("data/texas/texas.geojson")  # 254 counties
out = gdf.to_crs("EPSG:6933")  # world cylindrical equal-area (m)

by_state = out.dissolve(by="STATE_NAME", aggfunc="sum")   # counties → state
by_state["area_km2"] = by_state.geometry.area / 1_000_000
print(by_state[["area_km2"]].round(0))
# →              area_km2
#   STATE_NAME
#   Texas       684886.0     (254 county polygons merged into 1)
```

> **Lesson — dissolve** is **groupby** for geometry; take areal measurements only in an equal-area CRS, after the merge.

Vector Recipe: Reading, Clipping, and Joining

The everyday vector pattern is *read polygons, build points, keep the points that fall inside, then label each point with the polygon it lands in*. Here we read Texas counties, turn an Airbnb listings CSV into a point layer, **clip** away stray geocodes outside the state, then **spatial-join** every listing to its county.

```
import geopandas as gpd
import pandas as pd

# 1. Read bundled polygons: Texas counties (EPSG:4326)
cols = ["NAME", "STATE_NAME", "geometry"]
counties = gpd.read_file("data/texas/texas.geojson")[cols]

# 2. Build a point layer from a CSV of Airbnb listings (lon/lat columns)
df = pd.read_csv("data/texas/listings.csv.gz")
pts = gpd.GeoDataFrame(
    df, geometry=gpd.points_from_xy(df.longitude, df.latitude), crs="EPSG:4326"
)

# 3. CLIP: keep only the points that fall inside the Texas counties
inside = gpd.clip(pts, counties)
print(len(pts), "->", len(inside))   # 5835 -> 5834 (one dropped)
```

Vector Recipe (continued)

With the points clipped to Texas, the final step is the **spatial join**: attach to each listing the county polygon it falls within, then count listings per county.

```
# 4. JOIN: attach to each listing the county it sits in
listings_by_county = gpd.sjoin(
    inside, counties, how="inner", predicate="within")
print(listings_by_county["NAME"].value_counts().head(3))
# → Travis        5792   (Austin)
#   Williamson      35
#   Hays             7
```

> **Lesson —** clip, then **`sjoin(predicate="within")`**: the vector workhorse that tags each point with the polygon containing it in one pass.

> **Notebook:** notebooks/00_vector_data.ipynb

Vector Data: Further Learning

- **GeoPandas Getting Started**: The official entry point.
  - https://geopandas.org/en/stable/getting_started.html
- **Plotting with GeoPandas**: Map layers directly from GeoDataFrames.
  - https://geopandas.org/en/stable/docs/user_guide/mapping.html
- **Tutorial on Spatial Joins**: A fundamental concept explained well.
  - https://geopandas.org/en/stable/gallery/spatial_joins.html
- **Automating GIS Processes**: A great course with many vector data examples.
  - https://autogis-site.readthedocs.io/en/latest/

4. Raster Data

Working with Grids and Pixels

Raster data is a matrix of cells representing values over a continuous surface.

- **Key Properties**:
  - **Resolution**: The size of each pixel (e.g., 30 meters).
  - **Extent**: The bounding box of the entire grid.
  - **Bands**: A raster can have one or more layers (e.g., Red, Green, Blue bands in a color image).
  - **`nodata` value**: A special value assigned to pixels with no data.

- **Core Python Libraries**:
  - **`rasterio`**: The primary library for reading, writing, and manipulating raster files.
  - **`rioxarray`**: Integrates **`rasterio`** with **`xarray`** for powerful, labeled multi-dimensional array analysis (great for time-series!).

Raster formats in practice

Not every GeoTIFF is equal: how it is tiled and whether it carries overviews decides whether you can stream a small window over the network or must download the whole grid. A few format choices matter in day-to-day work.

- GeoTIFF: general-purpose workhorse; supports overviews and compression.
- COG (Cloud Optimized GeoTIFF): tiled + overviews for HTTP range requests.
- Bands & dtypes matter (uint8 imagery vs float32 surfaces); mind **nodata**.

Library Guide: `rasterio`

rasterio is the GDAL-backed library for reading and writing raster files, exposing the affine transform, CRS, **nodata** value, and band metadata. It supports **windowed** reads, so analysis can load only the required pixels instead of the whole grid.
Always open inside a **with rasterio.open(...) as src:** block so the file handle closes cleanly, respect **nodata** (cast to float and mask it before computing statistics), and choose resampling by data type — nearest for categorical labels, bilinear or cubic for continuous surfaces.

Library Guide: **rioxarray**

rioxarray combines Rasterio with **xarray**: labeled, N-dimensional arrays with a CRS, data-cube workflows across bands and time, and lazy Dask-backed computation for data too large to fit in memory. Its **.rio** accessor adds geospatial methods (**reproject**, **reproject_match**, **clip**, **to_raster**) to ordinary array math.
Use it for raster stacks, time series, and multi-band analysis. Before arithmetic between rasters, align grids with **reproject_match** and document **nodata** handling.

Rasterio: open and windowed read

```
import os
import rasterio
from rasterio.windows import Window

ras_path = "data/ghsl/ghsl_sao_paulo.tif"
if not os.path.exists(ras_path):
  print("Raster not found:", ras_path)
else:
  with rasterio.open(ras_path) as src:
    window = Window(0, 0, 512, 512)
    arr = src.read(1, window=window)
    print(arr.shape, src.res)
```

> **Lesson —** windowed reads pull only the pixels you request (clipped to the grid), so a small analysis never loads the whole raster into memory.

> **Notebook:** notebooks/00_raster_data.ipynb

Resampling and reprojection notes

- Choose resampling by data type:
  - Nearest: categorical labels
  - Bilinear/Cubic: continuous surfaces
- Reproject with **`rds.rio.reproject()`** (the **`.rio`** accessor on a DataArray, see recipe slide) or Rasterio's **`reproject`** API.

Raster Reprojection with `rioxarray`

`rioxarray` simplifies raster reprojection by handling the complex warping calculations for you.

```
import os
import rioxarray

os.makedirs("outputs", exist_ok=True)
ras_path = "data/ghsl/ghsl_sao_paulo.tif"
if not os.path.exists(ras_path):
  print("Raster not found:", ras_path)
else:
  rds = rioxarray.open_rasterio(ras_path)
  rds_3857 = rds.rio.reproject("EPSG:3857")
  print("Reprojected CRS:", rds_3857.rio.crs)
  rds_3857.rio.to_raster("outputs/ghsl_sao_paulo_3857.tif")
```

Raster alignment and resolution

Raster analysis compares cells. Two rasters can share a CRS and still be incompatible if their grids do not align.

Check before analysis

- **Resolution**: cell size in x and y
- **Extent**: covered bounding box
- **Transform**: origin, pixel size, and rotation
- **Nodata**: missing-value code and mask

Why it matters

- Misaligned grids compare different places.
- Resampling can smooth or change categories.
- Zonal statistics depend on cell size and boundary alignment.
- Document the grid used for reported results.

Use `reproject_match` when one raster should be aligned to another before subtraction, ratios, masking, or model inputs.

Raster Recipe: Reading, Masking, and Zonal Stats

This recipe computes the mean raster value within each polygon. No bundled vector overlaps the GHSL São Paulo raster, so we split the raster's **own extent** into West/East test halves — guaranteed to intersect it.

```
import os
import geopandas as gpd
import rasterio
from rasterio.mask import mask
from shapely.geometry import box
import numpy as np

ras_path = "data/ghsl/ghsl_sao_paulo.tif"
```

> **Notebook:** notebooks/00_raster_data.ipynb

Raster Recipe: build test polygons

The polygons inherit the raster CRS. This keeps the mask operation in the same coordinate system as the raster grid.

```
if os.path.exists(ras_path):
  with rasterio.open(ras_path) as src:
    # West/East halves of the raster's extent (guaranteed to overlap)
    left, bottom, right, top = src.bounds
    mx = (left + right) / 2
    cells = {"W": box(left, bottom, mx, top),
             "E": box(mx, bottom, right, top)}
    polygons = gpd.GeoDataFrame(
      {"name": list(cells)}, geometry=list(cells.values()), crs=src.crs)
```

Raster Recipe (continued)

The loop stays **inside** the `with` block so `src` is still open while masking (closing the file first raises "Dataset is closed").

```
    # Iterate through each polygon to perform the mask
    results = []
    for geom in polygons.geometry:
      # Mask the raster with the polygon geometry
      out_image, _ = mask(src, [geom], crop=True)

      # Mean of the valid (non-nodata) pixels, computed safely
      arr = out_image.astype('float32')
      nodata = src.nodata
      if nodata is not None:
        arr[arr == nodata] = np.nan
      mean_val = np.nanmean(arr)
      results.append(float(mean_val) if np.isfinite(mean_val) else None)

    polygons['mean_raster_value'] = results
    print(polygons[['name', 'mean_raster_value']])
```

Raster Recipe — the zonal-stats result

mask clips the raster to each polygon; **nanmean** reduces the valid pixels to one number per feature:

name	mean_raster_value
W	210.10
E	243.21

> **Lesson —** zonal statistics = mask by geometry, then reduce (mean / sum / …): the bridge from a pixel grid to one value per vector feature.

Raster Data: Further Learning

- **Rasterio Documentation & Cookbook**: Core reference for raster I/O and metadata.
  - https://rasterio.readthedocs.io/en/latest/
- **rioxarray Usage Guide**: Labeled multi-dimensional raster workflows.
  - https://corteva.github.io/rioxarray/stable/
- **Rasterio Quickstart Guide**: Basic raster reading, writing, and transformation.
  - https://rasterio.readthedocs.io/en/latest/quickstart.html
- **Cloud-Optimized GeoTIFFs (COGs)**: HTTP-range-readable raster storage for cloud and web workflows.
  - https://www.cogeo.org/

5. Network Data

Working with Nodes, Edges, and Connectivity

Spatial networks are graphs where nodes and/or edges have geographic locations. They are fundamental for routing, accessibility analysis, and logistics.

- **Use Cases**:
  - Calculating the shortest path between two points.
  - Finding all locations reachable within a 15-minute drive (isochrones).
  - Identifying the most critical intersections in a city.

- **Core Python Libraries**:
  - **OSMnx**: Downloads, analyzes, and visualizes street networks from OpenStreetMap.
  - **NetworkX**: A general-purpose library for graph analysis (centrality, pathfinding, etc.). **OSMnx** uses it under the hood.

Library Guide: `osmnx`

`osmnx` turns OpenStreetMap into a ready-to-analyze `networkx` graph in one call: `graph_from_place` downloads the streets, and it bundles geocoding, nearest-node queries, projection, plotting, and `basic_stats`. It supports routing, accessibility, and urban-form metrics without hand-parsing OSM.
Two practices matter: pick the `network_type` that matches your question (`drive` / `walk` / `bike` / `all`), and add edge speeds and travel times *before* routing on time. Cache graphs as GraphML so you are not re-downloading from the OSM servers on every run.

OSMnx essentials

Most OSMnx workflows combine the same operations: download, filter by travel mode, plot, summarize, and save the graph for reproducibility.

- Download graphs from OSM, filter by mode (drive/walk/bike), plot in 1 line.
- Compute stats (**basic_stats**), simplify topology, save/load graphs.

```
import osmnx as ox
G = ox.graph_from_place("Kreuzberg, Berlin, Germany", network_type="walk")
ox.plot_graph(G, node_size=0, edge_color="gray")
```

Library Guide: `networkx`

networkx is the general-purpose graph library that OSMnx builds on: it provides graph types and algorithms for shortest paths, centrality, connectivity, and flows independent of where the graph came from. It is the right layer when the research question is about topology rather than geometry.

Street networks are directed and have parallel edges, so they live in a **MultiDiGraph**; store edge weights such as **length** or **travel_time** consistently and document their units, since the same **shortest_path** call gives very different routes depending on which weight you pass.

NetworkX essentials

Once OSMnx hands you a graph, the analysis is pure NetworkX. The same algorithms work on any graph — here is the canonical shortest-path call on a toy example.

```
import networkx as nx
G = nx.path_graph(5)
print(nx.shortest_path(G, 0, 4))  # [0,1,2,3,4]
```

Project your graph for metrics

```
import osmnx as ox
G = ox.graph_from_place("Kreuzberg, Berlin, Germany", network_type="drive")
Gp = ox.project_graph(G)  # meters-based CRS for accurate lengths
u, v, k, data = list(Gp.edges(keys=True, data=True))[0]
print(data.get("length"))
```

Routing with travel time

To route on *time* rather than distance, let OSMnx 2.x impute a speed limit per edge (**add_edge_speeds**) and derive **travel_time** from it (**add_edge_travel_times**). The subtle bug to avoid: **geocode** returns **(lat, lon)** in **degrees**, so the nodes you snap to must come from a graph in the **same** units — we keep **G** unprojected and pass **X=lon, Y=lat**.

```
import osmnx as ox

G = ox.graph_from_place("Kreuzberg, Berlin, Germany", network_type="drive")
G = ox.routing.add_edge_speeds(G)   # km/h from OSM maxspeed+highway
G = ox.routing.add_edge_travel_times(G)  # travel_time = length/speed

# geocode -> (lat, lon); snap on the SAME (unprojected) graph, X=lon, Y=lat
orig = ox.geocoder.geocode("Görlitzer Bahnhof, Berlin")
dest = ox.geocoder.geocode("Kottbusser Tor, Berlin")
orig_n = ox.distance.nearest_nodes(G, X=orig[1], Y=orig[0])
dest_n = ox.distance.nearest_nodes(G, X=dest[1], Y=dest[0])

route = ox.routing.shortest_path(G, orig_n, dest_n, weight="travel_time")
print(len(route), "nodes")   # → ~12 nodes (illustrative; OSM data drifts)
```

OSM tags and filters (1/2)

OSM encodes everything in tags, so you rarely want *all* the edges. A custom filter lets you keep only the road classes your question is actually about.

- Use **custom_filter** in **graph_from_place** / **graph_from_polygon** to restrict tags.
- Example: only primary/secondary roads; exclude private/service roads.

OSM tags and filters (2/2)

The network you download is a modeling choice, not a fixed fact about the city — so sanity-check it before you trust any route or statistic computed on it.

- Align network type with your analysis (walk vs bike vs drive).
- Validate assumptions by visual inspection and simple stats.

Network Recipe: Get a Street Network with OSMnx

This example downloads the street network for a neighborhood, calculates basic stats, and plots it.

```
import osmnx as ox

# 1. Define a place and download the street network
place_name = "Kreuzberg, Berlin, Germany"
# You can specify network_type as 'drive', 'walk', 'bike', 'all'
G = ox.graph_from_place(place_name, network_type='drive')

# 2. Plot the network
fig, ax = ox.plot_graph(G, node_size=0, edge_linewidth=0.5, edge_color='gray')

# 3. Calculate basic statistics
stats = ox.stats.basic_stats(G)
print(f"Total street length: {stats['street_length_total'] / 1000:.2f} km")
```

Network Recipe (continued)

```
# 4. Find the shortest path (example)
origin = ox.geocoder.geocode("Görlitzer Bahnhof, Berlin")
destination = ox.geocoder.geocode("Kottbusser Tor, Berlin")
origin_node = ox.distance.nearest_nodes(G, origin[1], origin[0])
dest_node = ox.distance.nearest_nodes(G, destination[1], destination[0])

route = ox.routing.shortest_path(G, origin_node, dest_node, weight='length')
fig, ax = ox.plot_graph_route(G, route, node_size=0, bgcolor="k")
```

> **Lesson —** OSMnx turns a place name into a routable graph: geocode → snap to nearest nodes → shortest-path on edge **length**.

Network Data: Further Learning

- **OSMnx Examples Gallery**: Tutorials and advanced use cases.
  - https://github.com/gboeing/osmnx-examples
- **NetworkX Tutorial**: Learn the fundamentals of graph theory and analysis.
  - https://networkx.org/documentation/stable/tutorial.html
- **Python GIS: Network Analysis**: A practical book combining theory and code.
  - https://pythongis.org/part3/chapter-11/nb/00-introduction-to-spatial-network-analysis.html

6. Working Environment & Wrap-up

Reproducible working environment

The notebooks use a project-local Python environment with the packages listed in **notebooks/requirements.txt** and **notebooks/requirements-extras.txt**.

```
python -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r notebooks/requirements.txt
pip install -r notebooks/requirements-extras.txt
jupyter lab
```

Keep installation troubleshooting outside the lecture deck. The slide deck defines the spatial concepts and workflows; the setup guide documents platform specific fixes for GDAL, Rasterio, Fiona, Jupyter kernels, and Apple Silicon.

Summary

- A CRS defines how coordinates become positions, units, and valid measurements.
- Vector, raster, and network data support different operations and algorithms.
- GeoPandas provides the in-memory Python layer for vector spatial data science.
- Rasterio and rioxarray handle gridded data and metadata-aware raster workflows.
- OSMnx and NetworkX represent transport and accessibility as graph problems.

Deck 03 moves from in-memory analysis to persistent, indexed, and scalable spatial data management.